Building a Playwright POM Framework with Claude Web and CLI

Assignment Report

Project: Playwright POM Framework
Test Website: Sauce Demo
Automation Tool: Playwright Test
Browser: Chromium
Language: JavaScript (CommonJS)

---

1. Objective

The objective of this assignment was to build a reusable Playwright automation framework using the Page Object Model (POM) design pattern, with assistance from Claude Web. The framework was then extended to support data-driven testing using JSON test data and executed through the Playwright CLI.

2. Project Structure

The project was organized into separate folders to keep page objects, test cases, test data, and report evidence manageable.

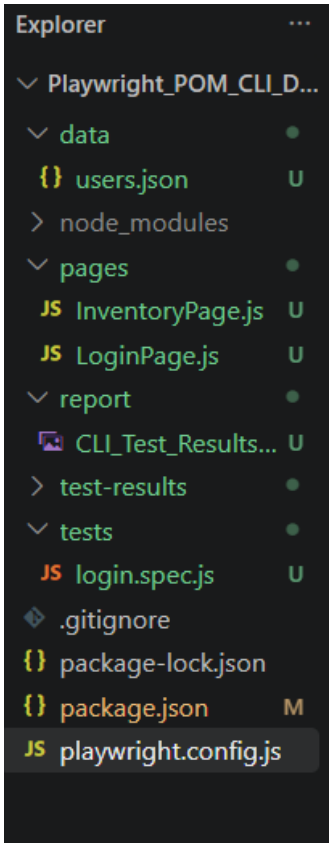


Figure 1: Project Structure

Key files and folders:

- pages/ – Contains the LoginPage and InventoryPage page objects.
- tests/ – Contains the Playwright login test.
- data/ – Contains the JSON test data.
- report/ – Contains screenshots and the assignment report.
- playwright.config.js – Contains Playwright test configuration.
- package.json – Contains project dependencies and the test command.

3. Page Object Model Implementation

Two-page object classes were created:

LoginPage.js

- Stores the username, password, login button, and error message locators.
- Provides methods to open the login page, enter credentials, submit the login form, and retrieve error messages.

InventoryPage.js

- Stores the Products page title and inventory list locators.
- Provides an isLoaded() method to verify that the inventory page has loaded.

The locators were checked against the actual Sauce Demo website using Chrome Developer Tools.

4. Data-Driven Testing

A JSON file named users.json was created to store three login scenarios:

Username	Password	Expected result
standard_user	secret_sauce	success
locked_out_user	secret_sauce	error
standard_user	wrong_password	error

The test file imports this JSON data and uses forEach() to generate a separate Playwright test for each record.

Successful login is verified by checking navigation to the inventory page. Failed login scenarios are verified by checking that an error message is visible and contains text.

5. Execution and Results

The tests were executed using the following command:

npm run test:login

Final result: 3 tests passed.

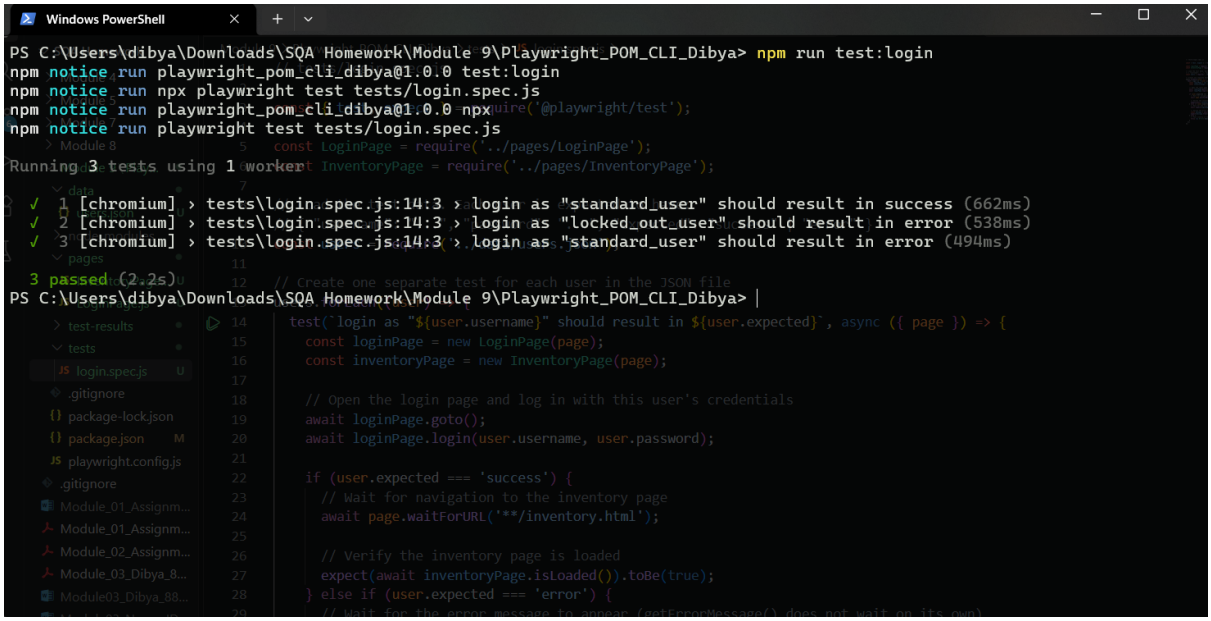


Figure 2: CLI Passed Screenshot

All three scenarios passed:

1. Standard user successfully logged in.
2. Locked-out user received the expected error.
3. Incorrect password produced the expected error.

6. Claude Web Assistance and Manual Improvements

Claude Web generated the initial Playwright configuration, Page Object classes, login test, and JSON-driven test implementation. I manually verified the website locators using Chrome Developer Tools. During debugging, I also improved the getErrorMessage() return value with .trim() and updated InventoryPage.isLoaded() to wait for the page title to become visible instead of relying on an immediate visibility check. These adjustments helped make the test assertions more reliable.

7. Conclusion

The assignment successfully demonstrated the implementation of a reusable Playwright POM framework and data-driven testing using JSON. All three login scenarios passed in Chromium through the Playwright CLI.